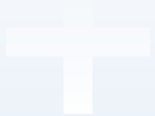


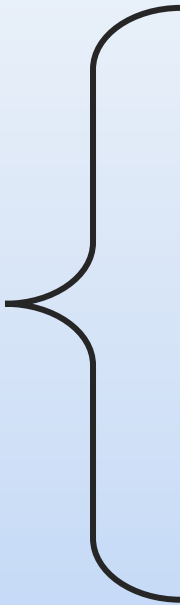
L1: Module 2

Himanshu Arya





TASKS



Go Lang

Git

Code Standards And Style Guide

Go Lang API Task

GO LANG

What is Go?

- Go is Open-Source General-Purpose programming language.
- It was developed in 2007 by **Robert Griesemer**, **Rob Pike**, and **Ken Thompson** at Google but launched in 2009.
- Solution to "Big Problems" at **Google**.

Go Features:

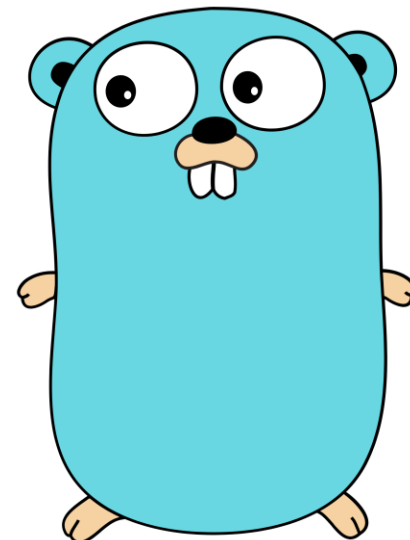
- Simplicity – Go is designed to be easy to learn and use.
- Garbage Collection - It has Automatic memory management.
- Fast Compile Times.
- Concurrency & Routines
- Go has a large and growing community of developers.

The Go Tool:

The Go tool is the standard tool for building, testing, and installing Go Program.

Compile and Run hello.go file -:

```
$ go run hello.go
```



GO LANG

Go Structure:

- Package declaration
- Import statements
- Variables
- Statements and Expressions
- Functions
- Comments

Go Datatypes:

- Basic type:
 - *Numbers, Strings, and Booleans.*
- Aggregate type:
 - *Array and structs.*
- Reference type:
 - *Pointers, slices, maps, functions, and channels.*
- Interface type



GO LANG

Go Variables

- var <variable_name> <type>

Go Conditional Statements

- If statements
- If-else statements
- If else-if statements
- Switch
- Multiple Case Switch

Go Control Statements

- for loop
- for range loop
- Go does not have while loops

Go Frameworks

- **GIN** – It is a HTTP web Framework written in Go.
- **IRIS**- It is an Express.js equivalent web framework that is easier to use for people coming from the Node.js community.



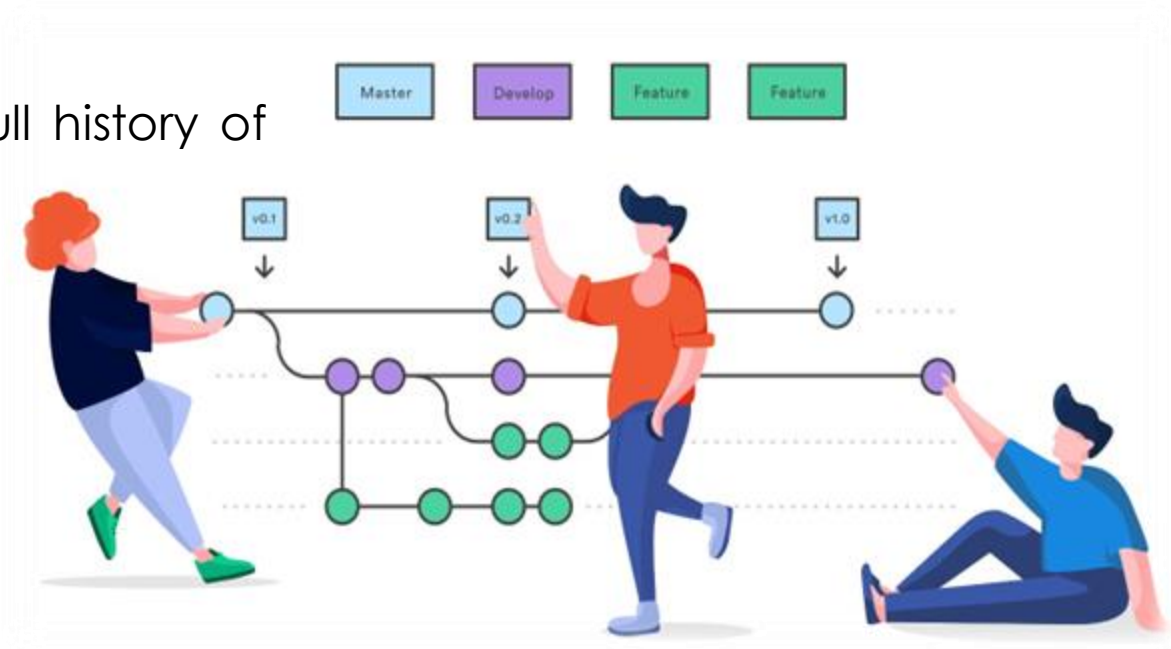
GIT

What is a 'Version Control System' ?

- A way to manage file and directories.
- Tracks changes over time.
- Recall previous version.

What is GIT ?

- Git is a **free** and **open-source** version control system, originally created by **Linus Torvalds** in 2005.
- Git is distributed: every developer has the full history of their code repository locally.
- No need to connect to Central server.
- Can work without internet connection.
- Git tracks **changes**, not versions.



GIT COMMANDS

1. Initialize a new project in directory:
 - **\$ git init**
2. Add a file using a text editor to the directory.
3. Add every change to the repo.
 - **\$ git add .**
4. Check status.
 - **\$ git status**
5. Commit the changes to the repo.
 - **\$ git commit -m "Initial Commit Message Here!!"**
6. To see what you have done.
 - **\$ git log**



```
xs535-himary@xs-host603-011:~/Downloads/Git$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/xs535-himary/Downloads/Git/.git/
xs535-himary@xs-host603-011:~/Downloads/Git$ git add .
xs535-himary@xs-host603-011:~/Downloads/Git$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   firstfile.go

xs535-himary@xs-host603-011:~/Downloads/Git$ git commit -m "Initial commit."
[master (root-commit) 6f2b5b5] Initial commit.
 1 file changed, 7 insertions(+)
 create mode 100644 firstfile.go
xs535-himary@xs-host603-011:~/Downloads/Git$ git log
commit 6f2b5b5faf20e38d73534c462462ede598cd6ada (HEAD -> master)
Author: dexteromen <himanshu@xenonstack.com>
Date:   Mon Feb 24 16:23:43 2025 +0530

    Initial commit.
xs535-himary@xs-host603-011:~/Downloads/Git$
```

GIT COMMANDS

Branch:

A branch represents an independent line of development.

Check Branch of the repo.

- \$ **git branch**

Create a new Branch.

- \$ **git branch new_branch_name**

Switching to new branch.

- \$ **git checkout new_branch_name**

Check the differences between the two branches.

- \$ **git diff first_branch..second_branch**

Stash Area:

Temporarily shelves (or *stashes*) changes you've made to your working copy so you can work on something else and then come back and re-apply them later.

- \$ **git stash save "stash_name"**
- \$ **git stash pop**
- \$ **git clear**



```
xs535-himary@xs-host603-011:~/Downloads/Git$ git branch
* master
xs535-himary@xs-host603-011:~/Downloads/Git$ git checkout --orphan new_branch
Switched to a new branch 'new_branch'
xs535-himary@xs-host603-011:~/Downloads/Git$ git rm -rf .
rm 'firstfile.go'
xs535-himary@xs-host603-011:~/Downloads/Git$ git add .
xs535-himary@xs-host603-011:~/Downloads/Git$ git commit -m "Initial Commit in new branch"
[new branch (root-commit) 1b7cc37] Initial Commit in new branch
1 file changed, 7 insertions(+)
create mode 100644 New_file.go
```

```
xs535-himary@xs-host603-011:~/Downloads/Git$ git add .
xs535-himary@xs-host603-011:~/Downloads/Git$ git status
On branch new_branch
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   stash.txt

xs535-himary@xs-host603-011:~/Downloads/Git$ git stash save "stash1"
Saved working directory and index state On new_branch: stash1
xs535-himary@xs-host603-011:~/Downloads/Git$ git stash list
stash@{0}: On new_branch: stash1
xs535-himary@xs-host603-011:~/Downloads/Git$ git stash pop
On branch new_branch
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   stash.txt

Dropped refs/stash@{0} (167bf6b3f286022646c768cba7a11c1855ae0c04)
xs535-himary@xs-host603-011:~/Downloads/Git$ git stash clear
```

CODE STANDARDS AND STYLE GUIDE

Indentation: Proper and consistent indentation is essential

Inline comments: Inline comments analyse the functioning of the subroutine

Rules for limiting the use of global: These rules file what types of data can be declared global and what cannot.

Structured Programming: Structured (or Modular) Programming methods shall be used.

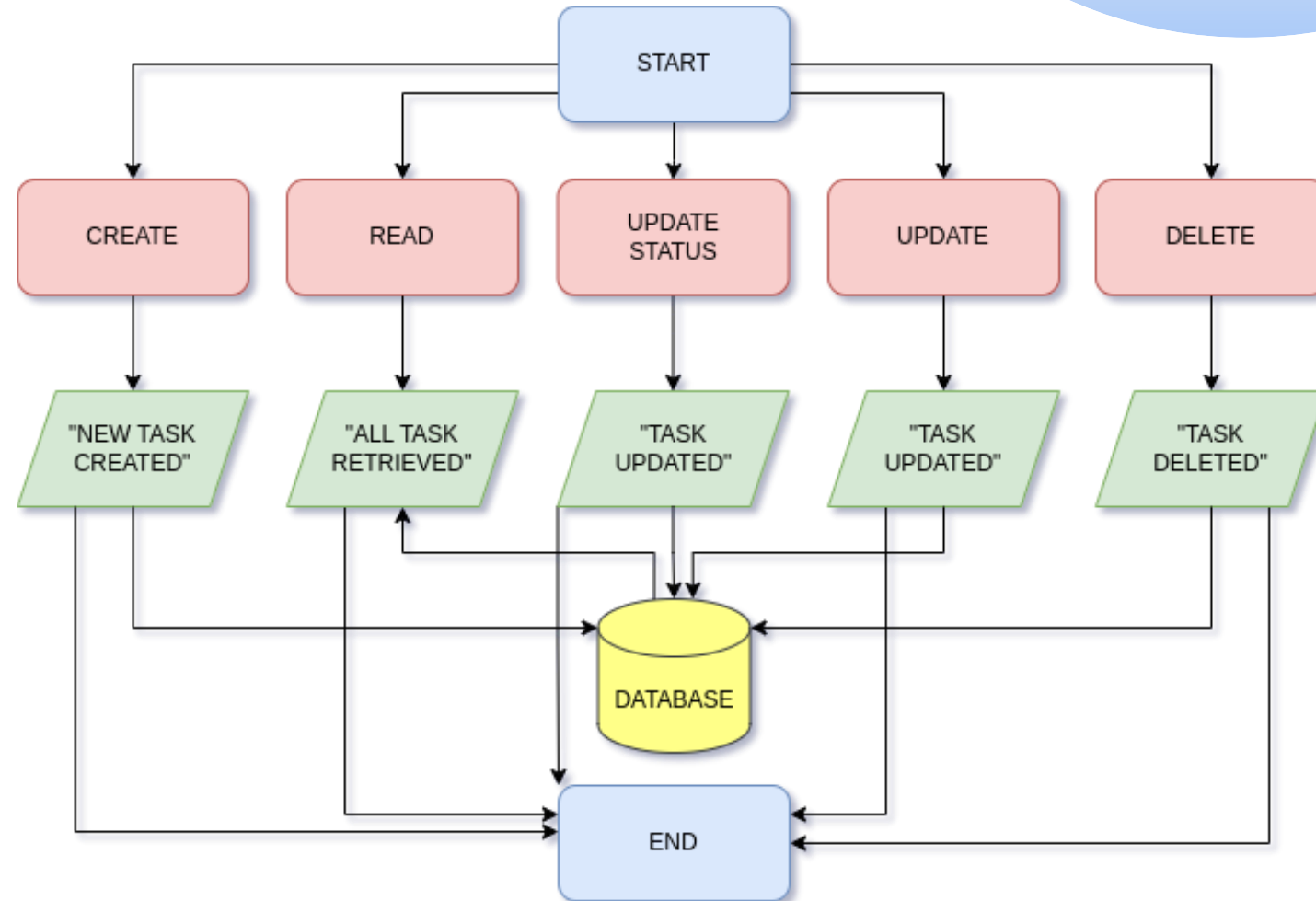
Naming conventions for global variables, local variables, and constant identifiers: A possible naming convention can be that global variable names always begin with a capital letter, local variable names are made of small letters, and constant names are always capital letters.

Error return conventions and exception handling system: Different functions in a program report the way error conditions are handled should be standard within an organization.



GO Lang API Task

The Work-Flow Diagram of the Task.





Thank You

Himanshu Arya

Associate Software Engineer

Himanshu@xenonstack.com